

INSTRUCTION SET ARCHITECTURES RESEARCH

Author: Kovalenko Elena

2026

Table of contents

Introduction.....	3
1 16-bit x86.....	4
1.1 General Characteristics.....	4
1.2 General Instruction Format.....	4
1.3 Multiplication Instructions.....	5
1.4 Division Instructions.....	6
2 Motorola 68000.....	8
2.1 General Characteristics.....	8
2.2 General Instruction Format.....	8
2.3 Multiplication Instructions.....	9
2.4 Division Instructions.....	10
3 PDP-11.....	12
3.1 General Characteristics.....	12
3.2 General Instruction Format.....	12
3.3 Multiplication Instructions.....	13
3.4 Division Instructions.....	14
Conclusion.....	16

Introduction

Instruction Set Architecture (ISA) defines the interface between hardware and software, specifying the set of machine instructions, their formats, addressing modes, and execution semantics. In the context of educational processors, 16-bit architectures occupy an important niche: they balance hardware simplicity with sufficient power to support non-trivial programs, including arithmetic-intensive workloads.

Historically, a wide range of 16-bit ISAs have been developed, each reflecting different design goals and constraints. Architectures such as PDP-11, MIL-STD-1750A, Motorola 68000 family, x86 (early generations), ARM (Thumb / early ARM), and AVR demonstrate diverse approaches to instruction encoding, operand specification, and arithmetic operation support. Despite their differences, these architectures share common challenges in designing efficient and compact arithmetic instructions — particularly multiplication and division, which are among the most complex integer operations to implement in hardware.

This document presents a comparative study of selected 16-bit instruction set architectures with a focus on:

- general instruction formats and operand models,
- semantic design of arithmetic instructions,
- and, in particular, the design of multiply and divide instructions.

The goal of this study is not only descriptive but also analytical. By examining how multiplication and division are specified and implemented across established ISAs, we derive design insights and trade-offs that directly influenced the development of multiplication and division instructions for the CdM-16 processor. This document therefore serves as a research artifact, forming the theoretical foundation for the design decisions made in the CdM-16 multiplication and division extensions.

1 16-bit x86

1.1 General Characteristics

The 16-bit x86 architecture represents one of the earliest and most influential Complex Instruction Set Computer (CISC) designs. This register-memory architecture uses little-endian byte order and follows an accumulator-oriented arithmetic model for certain operations, particularly multiplication and division.

Register file:

- General-purpose registers: AX, BX, CX, DX
- Index and pointer registers: SI, DI, BP, SP
- Segment registers: CS, DS, ES, SS
- Instruction pointer and status: IP, FLAGS

Memory model: segmented, with logical addresses formed as
segment:offset $\rightarrow$ physical address = (segment $\ll$ 4) + offset

Instruction length: variable (1 to 6 bytes)

1.2 General Instruction Format

x86 instructions are variable-length and consist of several optional fields:

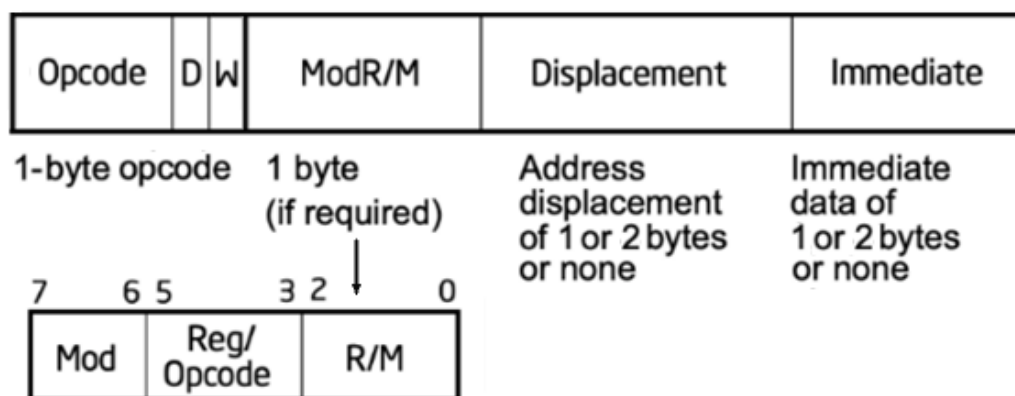


Figure 1 – Intel 8086 General Instruction Format

- Opcode – specifies the 6-bit operation code, direction (destination/source) of the operand specified by 'd' bit, width of the operand specified by 'w' bit;

- ModR/M – encodes the addressing mode and register operands.

The MOD field shows the mode of addressing. The register represented by the Reg field is one of the operands. The R/M field specifies another register or memory location i.e. the other operand;

- Optional displacement and immediate fields – used for memory addressing and constant operands, respectively.

This flexible encoding scheme allows x86 to support a wide variety of addressing modes at the cost of increased decoding complexity.

1.3 Multiplication Instructions

x86 provides implicit-operand multiplication instructions:

Instruction	Meaning	Sign
MUL r/m8	$AX \leftarrow AL \times r/m8$	Unsigned
MUL r/m16	$DX:AX \leftarrow AX \times r/m16$	Unsigned
IMUL r/m8	$AX \leftarrow AL \times r/m8$	Signed
IMUL r/m16	$DX:AX \leftarrow AX \times r/m16$	Signed

A key property of these instructions is that one operand is always implicit, either AL or AX. The result width is always twice the operand width, and the destination is a fixed register or register pair.

Encoding of MUL/IMUL

The MUL and IMUL instructions for 16-bit operands use the opcode byte F7. The specific operation is selected via the Reg field of the ModR/M byte, where 100 corresponds to MUL and 101 corresponds to IMUL.

Opcode	D	W	MOD	REG	R/M
111101	1	1	mm	100 or 101	rrr

Figure 2 – Intel 8086 MUL/IMUL Instruction Format

Semantics of Multiplication

For MUL r/m16: $DX:AX = AX \times r/m16$	For IMUL r/m16: $DX:AX = \text{sign_extend}(AX) \times \text{sign_extend}(r/m16)$
---	--

For the instruction MUL r/m16, the processor performs an unsigned multiplication of AX by the specified operand, storing the full 32-bit result in DX:AX. For IMUL r/m16, both operands are sign-extended before multiplication, and the signed 32-bit result is again stored in DX:AX. In both cases, the lower 16 bits of the result are placed in AX, while the upper 16 bits are placed in DX.

Overflow Handling and Flags Affected

The x86 architecture adopts a full-precision result strategy in which the result is always computed at double width. Overflow is detected by examining the high half of the product. For unsigned multiplication, overflow is indicated if DX is not zero, while for signed multiplication, overflow occurs if DX is not a sign-extension of AX. In such cases, the condition flags CF and OF are set, while flags AF, PF, SF, ZF are left undefined.

1.4 Division Instructions

Division is also implicit-operand and fixed-register.

Instruction	Meaning	Sign
DIV r/m8	$AX \div r/m8$	Unsigned
DIV r/m16	$DX:AX \div r/m16$	Unsigned
IDIV r/m8	$AX \div r/m8$	Signed
IDIV r/m16	$DX:AX \div r/m16$	Signed

Encoding of DIV / IDIV

The division instructions share the same opcode byte F7 as multiplication. 110 corresponds to DIV and 111 corresponds to IDIV in the Reg field.

Opcode	D	W	MOD	REG	R/M
111101	1	1	mm	110 or 111	rrr

Figure 3 – Intel 8086 DIV/IDIV Instruction Format

Semantics of Division

Instruction	Dividend Registers	Divisor	Quotient	Remainder	Signed
DIV r/m8	AX	r/m8	AL	AH	No
DIV r/m16	DX:AX	r/m16	AX	DX	No
IDIV r/m8	AX	r/m8	AL	AH	Yes
IDIV r/m16	DX:AX	r/m16	AX	DX	Yes

The DIV instruction performs unsigned division, while IDIV performs signed division. In both cases, the dividend is implicit and stored in fixed registers, while the divisor stored in a register or memory. The result is split into a quotient and a remainder, which are placed in predefined registers.

For byte-sized division, the dividend is taken from the AX register. After execution, the quotient is stored in AL and the remainder in AH. For word-sized division, the dividend is a double-word value formed by the DX:AX register pair. In this case, the quotient is written to AX and the remainder to DX.

The signed variant IDIV follows the same register conventions as DIV, but interprets both the dividend and the divisor as signed values. The remainder produced by IDIV always has the same sign as the dividend.

Exception handling

Unlike multiplication, division instructions in x86 can raise exceptions. A Divide Error exception (#DE) occurs if the divisor is zero or if the computed quotient does not fit into the destination register. After execution of DIV or IDIV, the state of the condition flags is undefined.

2 Motorola 68000

2.1 General Characteristics

The Motorola 68000 architecture, introduced in 1979, represents a classic orthogonal CISC design. Although internally the M68000 uses a 32-bit ALU and 32-bit registers, it is commonly classified as a 16-bit architecture due to its 16-bit external data bus and 16-bit instruction word size. This register–memory architecture uses big-endian byte order. Unlike x86, arithmetic instructions are not accumulator-oriented and operate on explicitly specified operands.

Register file:

- Data registers: D0–D7 (used for arithmetic and logical operations)
- Address registers: A0–A7 (used for addressing, A7 serves as the stack pointer)
- Program counter and Status: PC, SR

Memory model: flat and linear, with a 24-bit address space allowing up to 16 MB of memory

Instruction length: variable (2 to 10 bytes), always aligned to 16-bit words

2.2 General Instruction Format

M68000 instructions are variable-length and consist of several optional fields:

SINGLE EFFECTIVE ADDRESS OPERATION WORD	IMMEDIATE OPERAND OR SOURCE EFFECTIVE ADDRESS EXTENSION	DESTINATION EFFECTIVE ADDRESS EXTENSION
2 bytes, specifies operation and modes	If any, 2 to 4 bytes	If any, 2 to 4 bytes

Figure 4 – Motorola 68000 General Instruction Format

- Single Effective Address Operation Word – first and mandatory word, defines the properties (opcode, operand size, addressing modes, register selection) of the instruction and determines how many additional words will follow;
- Immediate Operand or Source Effective Address Extension – encodes immediate operands or specifies addressing parameters for the source operand;
- Destination Effective Address Extension – specifies addressing mode parameters for the destination operand.

Due to the complexity of the instructions structure and addressing modes – resulting from the goal of achieving an orthogonal ISA – we restrict our analysis to the first instruction word only. This level of detail is sufficient for studying the general operation of multiplication and division instructions. Additional extension words mainly serve to refine operand address calculation and therefore do not affect the core arithmetic semantics of multiplication and division in the general case.

2.3 Multiplication Instructions

The Motorola 68000 provides explicit-operand multiplication instructions for both signed and unsigned arithmetic:

Instruction	Meaning	Result size	Sign
MULU <ea>, Dn	$Dn \leftarrow Dn \times \langle ea \rangle$	32-bit	Unsigned
MULS <ea>, Dn	$Dn \leftarrow Dn \times \langle ea \rangle$	32-bit	Signed

Both instructions multiply a 16-bit source operand by the lower 16 bits of a data register. The full 32-bit result is stored in the destination data register Dn. Unlike x86, both operands are explicitly specified.

Encoding of MULU/MULS

The MULU and MULS instructions use the opcode bits 1100 and 11. The specific operation is selected via the Sign field of the, where 0 corresponds to MULU and 1 corresponds to MULS.

				Effective Address	
Opcode	Dest. Reg	Sign	Opcode	Mode	Register
1100	rrr	0/1	11	mmm	rrr

Figure 5 – Motorola 68000 MULU/MULS Instruction Format

The Effective Address field specifies the source operand and supports multiple addressing modes. The destination is always a data register.

Semantics of Multiplication

For MULU:	For MULS:
-----------	-----------

$D_n = \text{zero_extend}(D_n.w) \times \text{zero_extend}(\langle ea \rangle.w)$	$D_n = \text{sign_extend}(D_n.w) \times \text{sign_extend}(\langle ea \rangle.w)$
---	---

The processor performs a full-precision multiplication, producing a 32-bit result. The lower and upper halves of the product are stored together in the destination register.

Overflow Handling and Flags Affected

The Motorola 68000 does not provide overflow detection for multiplication. Since the full 32-bit result is always preserved, arithmetic overflow is not considered an exceptional condition.

Condition codes are set as follows:

- N (Negative): set if the result is negative.
- Z (Zero): set if the result is zero.
- V (Overflow): always cleared.
- C (Carry): always cleared.

2.4 Division Instructions

The division instructions in the Motorola 68000 are also explicit-operand and register-based:

Instruction	Meaning	Sign
DIVU $\langle ea \rangle, D_n$	$D_n \div \langle ea \rangle$	Unsigned
DIVS $\langle ea \rangle, D_n$	$D_n \div \langle ea \rangle$	Signed

In contrast to multiplication, division operates on a 32-bit dividend stored in a data register and a 16-bit divisor.

Encoding of DIVU / DIVS

				Effective Address	
Opcode	Dest. Reg	Sign	Opcode	Mode	Register
1000	rrr	0/1	11	mmm	rrr

Figure 6 – Motorola 68000 DIVU/DIVS Instruction Format

Semantics of Division

For both DIVU and DIVS, the dividend is taken from the destination data register Dn as a 32-bit value. The divisor is a 16-bit operand specified by the effective address.

After execution:

- The quotient is placed in the lower 16 bits of Dn.
- The remainder is placed in the upper 16 bits of Dn.

For signed division, both operands are interpreted as signed integers, and the remainder has the same sign as the dividend.

Exception and Overflow Handling

Division instructions may generate exceptions. A division-by-zero exception occurs if the divisor is zero. Additionally, a division overflow exception is raised if the quotient does not fit into 16 bits.

Condition codes after division are defined as:

- N: reflects the sign of the quotient.
- Z: set if the quotient is zero.
- V: set on division overflow.
- C: always cleared.

3 PDP-11

3.1 General Characteristics

The PDP-11 is a 16-bit CISC architecture that strongly influenced later processor designs. This little-endian architecture follows a highly orthogonal register-memory model with uniform instruction formats and consistent addressing modes across most instructions. The PDP-11 instruction set is word-oriented, but also supports byte-level operations.

Register file:

- General-purpose registers: R0–R7 (R6 is the stack pointer (SP) and R7 is the program counter (PC));
- Status: Processor Status Word (PSW)

Memory model: flat 16-bit address space, allowing up to 64 KB of memory

Instruction length: variable (2 to 6 bytes)

3.2 General Instruction Format

PDP-11 instructions are variable-length and consist of several optional fields:

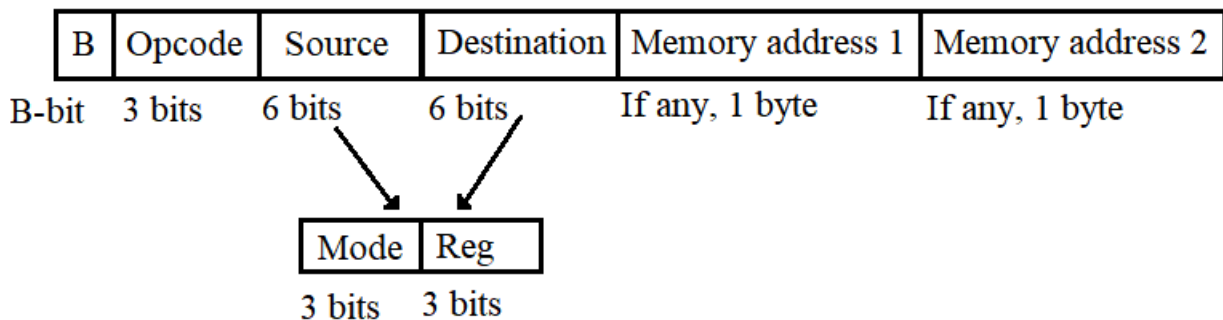


Figure 7 – PDP-11 General Double-Operand Instruction Format

B-bit – defines whether instruction operates with words (0) or bytes (1);

Opcode – specifies the 3-bit operation code for current instruction format;

Source and Destination – source and destination operands respectively. Each operand is specified using an addressing mode and a register field. The addressing modes include register, register deferred, auto-increment, auto-decrement, indexed, and immediate (via PC-relative addressing). This uniform encoding contributes to the high orthogonality of the ISA;

Memory address 1, Memory address 2 – used only when required by the selected addressing modes

3.3 Multiplication Instructions

The PDP-11 provides a single explicit signed multiplication instruction:

Instruction	Meaning	Sign
MUL src, dst	$\text{dst}:\text{dst}+1 \leftarrow \text{dst} \times \text{src}$	Signed

Encoding of MUL

Due to encoding limitations, some two-operand instructions utilize an addressing mode operand and a register operand. The source operand is specified using a full effective address field, though the destination operand is specified as a register only.

Opcode	Reg	Source
0111000	rrr	mmmmrrr

Figure 8 – PDP-11 MUL Instruction Format

Semantics of Multiplication

For MUL src, R_n (where n is even):

$$\text{Rn}:\text{Rn}+1 = \text{sign_extend}(\text{Rn}) \times \text{sign_extend}(\text{src})$$

The MUL instruction performs a signed 16-bit multiplication between the source operand and a destination register. Both operands are interpreted as signed 16-bit two's complement values. The processor computes the full 32-bit product. The destination register must be an even-numbered register, as the 32-bit result is stored across a register pair. The lower 16 bits of the result are placed in the destination register, while the upper 16 bits are placed in the next higher register.

The source operand may be specified using any valid addressing mode, while the destination is always a register.

Overflow Handling and Flags Affected

Processor Status Word condition codes affected:

- **N (Negative)** - set if product is < 0 , cleared otherwise;
- **Z (Zero)** - set if product is 0, cleared otherwise

- **V (Overflow)** — cleared;
- **C (Carry)** — set if the result is less than -2^{15} or greater than or equal to $2^{15}-1$.

Overflow detection is left to software by examining the high word of the result. This design follows a full-precision arithmetic strategy, but without explicit overflow flags.

3.4 Division Instructions

The PDP-11 provides an explicit-operand signed division instruction:

Instruction	Meaning	Sign
DIV src, dst	$\text{dst} \leftarrow (\text{dst}:\text{dst}+1) \div \text{src}$	Signed

Encoding of DIV

Opcode	Reg	Source
0111001	rrr	mmmrtrr

Figure 9 – PDP-11 DIV Instruction Format

Semantics of Division

For DIV src, Rn:

- Dividend: Rn:Rn+1 (32-bit signed)
- Divisor: src (16-bit signed)
- Quotient $\rightarrow$ Rn
- Remainder $\rightarrow$ Rn+1

The quotient must fit into a 16-bit signed value. The remainder has the same sign as the dividend. The dividend is a 32-bit signed value stored in a register pair (dst and dst+1). The destination register must be even-numbered and identifies the register pair holding both the dividend and the result. The divisor is a 16-bit signed value specified by the source operand.

Exception Handling and Flags (Division)

Unlike multiplication, division instructions in PDP-11 can cause traps:

- Division by zero

- Quotient overflow (when the result does not fit in 16 bits)

In such cases, the processor generates a trap, transferring control to an exception handler. After a successful division, the condition codes are set as follows:

- **N** — set if quotient < 0 , cleared otherwise;
- **Z** — set if quotient $= 0$, cleared otherwise;
- **V** — set if source $= 0$ or if the absolute value of the register is larger than the absolute value of the source;
- **C** — set if divide 0 attempted, cleared otherwise.

Conclusion

In this work, several 16-bit instruction set architectures were studied in order to choose a suitable approach for implementing multiplication and division in the educational load-store processor CdM16. The analysis focused on how these architectures encode instructions, how operands and results are handled, and how overflow and errors are processed.

Analysis of Architectural Solutions for CdM16

The 16-bit x86 (8086) architecture uses an implicit-operand model for multiplication and division. One operand and the destination registers are fixed (AX and DX). This makes instruction encoding compact, but strongly ties arithmetic operations to specific registers. Such behavior does not match the design principles of CdM16. In addition, this model complicates compiler support.

The Motorola 68000 architecture provides a more uniform register set and flexible addressing modes. The main limitation of this architecture for CdM16 is not implicit operands, but the fact that it is fundamentally 32-bit, with wide registers and complex effective addressing. This makes it unsuitable for direct adoption in a simple 16-bit educational processor.

The PDP-11 architecture follows a clean two-operand model and has a relatively simple instruction format. Nevertheless, its multiplication and division instructions use fixed register pairs for results. This adds control-flow complexity and reduces clarity for students.

Overall, these architectures show that while powerful arithmetic instructions are possible, they often introduce implicit behavior and special cases that conflict with the goals of CdM16.

Other Architectures Worth Mentioning

Several additional architectures were considered:

- MIL-STD-1750A uses explicit operands. Results are stored in general-purpose registers, and extended results may use register pairs. It is reliable, but too complex for an educational processor.
- AVR implements multiplication using two registers and stores the result in a fixed register pair. Division is usually implemented in software. This approach is simple, but still depends on implicit destination registers.
- ARM Thumb uses explicit register operands and stores results in general-purpose registers. Earlier versions lack hardware division. This design is simple, scalable, and works well with modern compilers.

These architectures highlight the advantages of explicit operands and predictable register usage.

Final Decision for CdM16

Based on this study, CdM16 uses two-operand multiplication and division instructions, where the result is written back to the source registers. The final instruction set includes:

- `mulu rl, rh`
- `muls rl, rh`
- `divu rl, rh`
- `divs rl, rh`

Here, `rl` and `rh` act both as input operands and as destination registers. This removes implicit registers, keeps the instruction behavior clear, and simplifies both hardware and assembler implementation.

The choice of a two-operand format was also influenced by instruction encoding constraints: only the `op2` instruction format had free opcode space for new instructions.

Signed and unsigned operations are implemented as separate instructions. This avoids ambiguity, simplifies hardware logic, and matches common ISA design practices.

Finally, this design makes future LLVM integration easier. LLVM works best with explicit, register-based operations, which fits well with the chosen CdM16 instruction format.

In conclusion, the selected design provides a good balance between simplicity, clarity, and extensibility, making it well suited for an educational load-store processor like CdM16.